

Assignment 18: Advanced Data Wrangling Techniques

Name Ayleen Santander

1. Loading dataset:

- Copy the code and run

```
import pandas as pd

# Define column names as per UCI Adult dataset description
column_names = [
    'age', 'workclass', 'fnlwgt', 'education', 'education_num', 'marital_status',
    'occupation', 'relationship', 'race', 'sex', 'capital_gain', 'capital_loss',
    'hours_per_week', 'native_country', 'income']

# Load the dataset from a URL
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
uci = pd.read_csv(url, names=column_names, na_values='?', skipinitialspace=True)

print("UCI Adult Dataset:")
print(uci.head())
```

UCI Adult Dataset:

	age	workclass	fnlwgt	education	education_num \	
0	39	State-gov	77516	Bachelors	13	
1	50	Self-emp-not-inc	83311	Bachelors	13	
2	38	Private	215646	HS-grad	9	
3	53	Private	234721	11th	7	
4	28	Private	338409	Bachelors	13	

	marital_status	occupation	relationship	race	sex \	
0	Never-married	Adm-clerical	Not-in-family	White	Male	
1	Married-civ-spouse	Exec-managerial	Husband	White	Male	
2	Divorced	Handlers-cleaners	Not-in-family	White	Male	
3	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	
4	Married-civ-spouse	Prof-specialty	Wife	Black	Female	

	capital_gain	capital_loss	hours_per_week	native_country	income	
0	2174	0	40	United-States	<=50K	
1	0	0	13	United-States	<=50K	
2	0	0	40	United-States	<=50K	
3	0	0	40	United-States	<=50K	
4	0	0	40	Cuba	<=50K	

Part 2 : Following the instructions and create the funtions and Methods

1. Simple grouping: Group by 'education' and calculate the mean age and mean hours per week. Use groupby and agg to calculate the mean values.

```
clone_uci = uci.copy()
grouped = clone_uci.groupby('education').agg({'age': 'mean', 'hours_per_week': 'mean'})
print(grouped)
```

	age	hours_per_week
education		
10th	37.429796	37.052519
11th	32.355745	33.925957
12th	32.000000	35.780600
1st-4th	46.142857	38.255952
5th-6th	42.885886	38.897898
7th-8th	48.445820	39.366873
9th	41.060311	38.044747
Assoc-acdm	37.381443	40.504217
Assoc-voc	38.553546	41.610709
Bachelors	38.904949	42.614006
Doctorate	47.702179	46.973366
HS-grad	38.974479	40.575374
Masters	44.049913	43.836332
Preschool	42.764706	36.647059
Prof-school	44.746528	47.425347
Some-college	35.756275	38.852284

2. Multiple grouping: Group by 'education' and 'sex', and calculate sum, count, and

- mean for 'hours_per_week'. Use groupby and agg with multiple aggregation functions.

```
clone_uci = uci.copy()
mult_grouped = clone_uci.groupby(['education','sex'], observed = True)[['age', 'hours_per_week']].agg(['mean', 'median', 'count'])
print(mult_grouped)
```

education	sex	age			hours_per_week		
		mean	median	count	mean	median	count
10th	Female	35.447458	31.0	295	32.111864	37.0	295
	Male	38.346395	36.0	638	39.336991	40.0	638
11th	Female	30.546296	24.0	432	29.821759	30.5	432
	Male	33.407806	30.0	743	36.312248	40.0	743
12th	Female	29.756944	23.0	144	31.791667	35.0	144
	Male	33.117647	29.0	289	37.768166	40.0	289
1st-4th	Female	48.521739	46.0	46	31.978261	37.5	46
	Male	45.245902	45.5	122	40.622951	40.0	122
5th-6th	Female	44.369048	44.0	84	36.047619	40.0	84
	Male	42.385542	41.0	249	39.859438	40.0	249
7th-8th	Female	49.693750	52.0	160	36.200000	40.0	160
	Male	48.034979	49.0	486	40.409465	40.0	486
9th	Female	42.131944	42.0	144	33.916667	40.0	144
	Male	40.643243	38.0	370	39.651351	40.0	370
Assoc-acdm	Female	36.361045	35.0	421	37.358670	40.0	421
	Male	38.046440	37.0	646	42.554180	40.0	646
Assoc-voc	Female	37.872000	36.0	500	37.830000	40.0	500
	Male	38.939909	38.0	882	43.753968	40.0	882
Bachelors	Female	35.635578	33.0	1619	39.329216	40.0	1619
	Male	40.321734	39.0	3736	44.037473	40.0	3736
Doctorate	Female	45.325581	43.5	86	47.302326	40.0	86
	Male	48.327217	48.0	327	46.886850	45.0	327
HS-grad	Female	38.678171	37.0	3390	36.577286	40.0	3390
	Male	39.115736	37.0	7111	42.481367	40.0	7111
Masters	Female	43.074627	43.0	536	41.113806	40.0	536
	Male	44.490312	44.0	1187	45.065712	42.0	1187
Preschool	Female	41.750000	39.5	16	31.875000	35.0	16
	Male	43.228571	42.0	35	38.828571	40.0	35
Prof-school	Female	40.195652	39.5	92	44.793478	40.0	92
	Male	45.611570	44.0	484	47.925620	50.0	484
Some-college	Female	33.740556	30.0	2806	34.574840	40.0	2806
	Male	37.017391	35.0	4485	41.528428	40.0	4485

3. Applying functions: Group by 'marital_status' and apply a custom function to find the range of 'age'. Use groupby and apply with a custom function.

-Explanation parameters

- include_groups = False = just tells pandas: "don't hand the maritalstatus column to my function , only care about the rest.

```
#First option for this code
clone_uci = uci.copy()

def age_range(group):
    return group['age'].max() - group['age'].min()

result = clone_uci.groupby('marital_status', observed=True).apply(age_range, include_groups = False)
print(result)
```

```
marital_status
Divorced          72
Married-AF-spouse  56
Married-civ-spouse 73
Married-spouse-absent 62
Never-married     73
Separated         72
Widowed          72
dtype: int64
```

```
#OPTIONAL Second option of the code
def age_range(x):
    return x.max() - x.min()

grouped = clone_uci.groupby(['marital_status', 'relationship'], observed=True)['age'].apply(age_range)
print(grouped)
```

marital_status	relationship	
Divorced	Not-in-family	64
	Other-relative	53
	Own-child	53
	Unmarried	71
Married-AF-spouse	Husband	21
	Other-relative	0
	Own-child	0
	Wife	56
Married-civ-spouse	Husband	73
	Not-in-family	38
	Other-relative	60
	Own-child	33
	Wife	72
Married-spouse-absent	Not-in-family	60
	Other-relative	43
	Own-child	30
	Unmarried	53
Never-married	Not-in-family	73
	Other-relative	57
	Own-child	73
	Unmarried	63
Separated	Not-in-family	70
	Other-relative	46
	Own-child	72
	Unmarried	53
Widowed	Not-in-family	65
	Other-relative	54
	Own-child	37
	Unmarried	63

Name: age, dtype: int64

4. Group transforms: Group by 'occupation' and transform to find the mean age within each group. Use groupby and transform to add a new column with transformed data.

```
# Define a function to perform the transformation
def find_age_mean(group):
    return group.mean()
# Apply the transformation using groupby and transform
average_g=clone_uci['Group_Average'] = clone_uci.groupby('occupation')['age'].transform(find_age_mean)

print(clone_uci[['occupation', 'age', 'Group_Average']])
```

	occupation	age	Group_Average
0	Adm-clerical	39	36.964456
1	Exec-managerial	50	42.169208
2	Handlers-cleaners	38	32.165693
3	Handlers-cleaners	53	32.165693
4	Prof-specialty	28	40.517633
...	...	...	...
32556	Tech-support	27	37.022629
32557	Machine-op-inspct	40	37.715285
32558	Adm-clerical	58	36.964456
32559	Adm-clerical	22	36.964456
32560	Exec-managerial	52	42.169208

[32561 rows x 3 columns]

5.Creating a pivot table: Create a pivot table to show the mean 'hours_per_week' for each 'occupation' and 'sex'. Use pivot_table to create a pivot table with specified aggregation

```
pivot_table = clone_uci.pivot_table(
    values='hours_per_week',
    index='occupation',
    columns='sex',
    observed = True,
    aggfunc=['mean', 'count']
)
print(pivot_table)
```

	mean		count	
sex	Female	Male	Female	Male
occupation				
?	29.976219	33.525948	841.0	1002.0
Adm-clerical	36.741033	39.240065	2537.0	1233.0

Armed-Forces	NaN	40.666667	NaN	9.0
Craft-repair	39.869369	42.443642	222.0	3877.0
Exec-managerial	41.517688	46.371173	1159.0	2907.0
Farming-fishing	37.784615	47.634015	65.0	929.0
Handlers-cleaners	36.103659	38.198176	164.0	1206.0
Machine-op-inspct	38.929091	41.447658	550.0	1452.0
Other-service	33.437778	36.223411	1800.0	1495.0
Priv-house-serv	32.489362	39.875000	141.0	8.0
Prof-specialty	39.423762	44.096762	1515.0	2625.0
Protective-serv	38.526316	43.446771	76.0	573.0
Sales	34.274743	44.223712	1263.0	2387.0
Tech-support	37.295977	40.713793	348.0	580.0
Transport-moving	36.711111	45.130723	90.0	1507.0